

Berkeley Indices Trajectory Extractor (BITE) User Guide V1.0

Matlab scripts and BITE User Guide written by *Yanlei Chen*

yanleichen@berkeley.edu

yanleichen@gmail.com

The algorithm is introduced in,

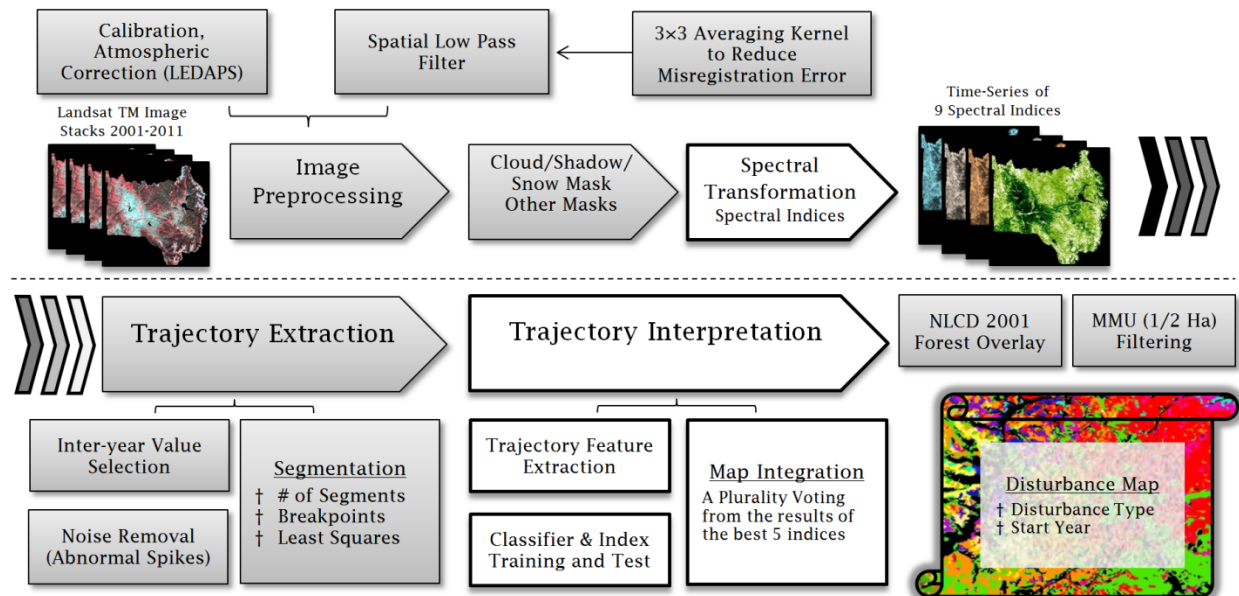
Chen, Y., Liang, L., Hawbaker, T.J., Gong, P., Biging, G.S., Zhu, Z., BITE: An algorithm for mapping slow-onset forest disturbances caused by mountain pine beetles with Landsat image stacks. *Remote Sensing of Environment*, submitted.

Table of Contents

1	Introduction	2
2	Data Preparation	3
3	Procedures.....	6
3.1	Processing subsets.....	6
3.2	Preprocessing Masking Files (Optional).....	7
3.3	Filtering and Masking.....	8
3.4	Calculating Spectral Indices	9
3.5	Stacking Time-Series Layers.....	10
3.6	Trajectory Extraction.....	11
3.7	Fitting Classification Models (Optional)	13
3.8	Classifying Images and Integrating Maps.....	15
3.9	Integrating Maps.....	17
3.10	Post-Classification Processing	18
4	Acknowledgement.....	20

1 Introduction

Berkeley Indices Trajectory Extractor (BITE) is an algorithm to derive disturbance maps from multi-temporal remote sensing image stacks. Particularly, BITE can distinguish persistent forest, slow-onset disturbances and rapid-onset disturbances. Slow-onset disturbances are usually caused by diseases and insects, which result in forest loss over a long period. Rapid-onset disturbances are abrupt loss of canopies that are usually caused by clearcut, logging, prescribed fires, wildfires and other natural disasters. BITE features a distinctive processing flow that requires almost no parameters for tuning but a training dataset to fit statistical learning models. The models are used to predict the features extracted from the trajectories, which are derived from segmenting time-series for multiple spectral indices. For each spectral index, a disturbance map can be derived, and by integrating these maps a final integrated disturbance map is produced via a plurality voting, thus is more accurate than any disturbance map of a single spectral index. BITE algorithm was tested to be resistant to data gaps (clouds/shadow/snow) and noises (haze, temporal fluctuation, minor misregistrations). An example of using Landsat TM images from 2001-2011 is presented in the flow chart below.



BITE is written in Matlab script language and is available for public access. This document is a guide for using the BITE source code to produce disturbance maps using satellite images.

2 Preparation

LIBSVM package is required. Please download at

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/#download>.

Extract the Matlab files into the same folder as BITE scripts and type “make” to compile mex files.

There are a few different types of necessary georeferenced images. The requirements of formatting of all the images are the same and they should all be:

1. ENVI Standard File
2. Interleave is BSQ
3. Identical map info (Coordinates System, Projection, XY Starting Location)
4. Except for the Subsetting process, all images should have the same spatial size (rows and lines).

The following data is required or optional based on what users want to achieve:

Landsat Surface Reflectance Images

The images should be derived during summer time for multiple years (best around 10 years). Users need to convert the images into **ENVI standard file** format using **BSQ** (Data sequenced band after band). Header files (.hdr) are required for each image. Image filenames should have no extensions. Users can either download Landsat L1T data then use LEDAPS tool in the Linux operation system to automatically convert the data into surface reflectance images or directly download the Landsat CDR Land Surface Reflectance data.

After converted into ENVI standard file, the filename must present its acquisition time in the format of yyyyddd. For instance, the image was acquired in 2011, Sep 9. And Sep 9 is the 252th day in the year so the file should be named as 2011252 (and 2011252.hdr for the header file). The lookup table for the Day of Year is shown below (http://disc.gsfc.nasa.gov/julian_calendar.shtml),

Day	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Day	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
1	001	032	060	091	121	152	182	213	244	274	305	335	1	001	032	061	092	122	153	183	214	245	275	306	336
2	002	033	061	092	122	153	183	214	245	275	306	336	2	002	033	062	093	123	154	184	215	246	276	307	337
3	003	034	062	093	123	154	184	215	246	276	307	337	3	003	034	063	094	124	155	185	216	247	277	308	338
4	004	035	063	094	124	155	185	216	247	277	308	338	4	004	035	064	095	125	156	186	217	248	278	309	339
5	005	036	064	095	125	156	186	217	248	278	309	339	5	005	036	065	096	126	157	187	218	249	279	310	340
6	006	037	065	096	126	157	187	218	249	279	310	340	6	006	037	066	097	127	158	188	219	250	280	311	341
7	007	038	066	097	127	158	188	219	250	280	311	341	7	007	038	067	098	128	159	189	220	251	281	312	342
8	008	039	067	098	128	159	189	220	251	281	312	342	8	008	039	068	099	129	160	190	221	252	282	313	343
9	009	040	068	099	129	160	190	221	252	282	313	343	9	009	040	069	100	130	161	191	222	253	283	314	344
10	010	041	069	100	130	161	191	222	253	283	314	344	10	010	041	070	101	131	162	192	223	254	284	315	345
11	011	042	070	101	131	162	192	223	254	284	315	345	11	011	042	071	102	132	163	193	224	255	285	316	346
12	012	043	071	102	132	163	193	224	255	285	316	346	12	012	043	072	103	133	164	194	225	256	286	317	347
13	013	044	072	103	133	164	194	225	256	286	317	347	13	013	044	073	104	134	165	195	226	257	287	318	348
14	014	045	073	104	134	165	195	226	257	287	318	348	14	014	045	074	105	135	166	196	227	258	288	319	349
15	015	046	074	105	135	166	196	227	258	288	319	349	15	015	046	075	106	136	167	197	228	259	289	320	350
16	016	047	075	106	136	167	197	228	259	289	320	350	16	016	047	076	107	137	168	198	229	260	290	321	351
17	017	048	076	107	137	168	198	229	260	290	321	351	17	017	048	077	108	138	169	199	230	261	291	322	352
18	018	049	077	108	138	169	199	230	261	291	322	352	18	018	049	078	109	139	170	200	231	262	292	323	353
19	019	050	078	109	139	170	200	231	262	292	323	353	19	019	050	079	110	140	171	201	232	263	293	324	354
20	020	051	079	110	140	171	201	232	263	293	324	354	20	020	051	080	111	141	172	202	233	264	294	325	355
21	021	052	080	111	141	172	202	233	264	294	325	355	21	021	052	081	112	142	173	203	234	265	295	326	356
22	022	053	081	112	142	173	203	234	265	295	326	356	22	022	053	082	113	143	174	204	235	266	296	327	357
23	023	054	082	113	143	174	204	235	266	296	327	357	23	023	054	083	114	144	175	205	236	267	297	328	358
24	024	055	083	114	144	175	205	236	267	297	328	358	24	024	055	084	115	145	176	206	237	268	298	329	359
25	025	056	084	115	145	176	206	237	268	298	329	359	25	025	056	085	116	146	177	207	238	269	299	330	360
26	026	057	085	116	146	177	207	238	269	299	330	360	26	026	057	086	117	147	178	208	239	270	300	331	361
27	027	058	086	117	147	178	208	239	270	300	331	361	27	027	058	087	118	148	179	209	240	271	301	332	362
28	028	059	087	118	148	179	209	240	271	301	332	362	28	028	059	088	119	149	180	210	241	272	302	333	363
29	029		088	119	149	180	210	241	272	302	333	363	29	029	060	089	120	150	181	211	242	273	303	334	364
30	030		089	120	150	181	211	242	273	303	334	364	30	030		090	121	151	182	212	243	274	304	335	365
31	031		090		151		212	243		304		365	31	031		091		152		213	244		305		366

Cloud Masks (Optional)

The mask images should be derived from Fmask algorithm or downloaded as a single band with the Landsat CDR surface reflectance file. The cloud mask file should also be converted into ENVI standard file format and named as yyyydddFmask and yyyydddFmask.hdr.

Study Area Mask (Optional)

The study area must be named exactly as “studyarea” and “studyarea.hdr”. It only has one band, in which the data value 0 means off and masked, while 1 is on and retained.

Supplemental Mask (Optional)

This is an additional mask either manually drawn or automatically generated for other purposes. It is named as “yyyydddsm” and “yyyydddsm.hdr”. It only has one band, in which the data value 0 means off and masked, while 1 is on and retained.

Forest Mask

This file has one band, in which the data value 0 means off and masked, while 1 is on and retained. The naming of this file is flexible, but it should have the same spatial size as the time-series images.

Training Data Samples (Optional)

If the user intends to build his or her own models, the training data samples are required. Prepare three shape files for persistent forest, slow-onset disturbances and rapid-onset disturbances respectively as training data samples. The samples are in point format. They must be named as

'persistent.shp', 'slowonset.shp' and 'rapidonset.shp'. The shape files should include all files with the same base name, with extension such as ".dbf", ".shx", ".prj", ".sbn" and ".sbx". The projection should as be identical to the time-series images.

Default Classification Models (Included in the package)

The package includes the default classification models for all spectral indices and two classification algorithms. The 301 training data samples were selected in Grand County, Colorado. However, it must be noted that the generalization of the default models has not been experimented to other areas yet. Use with cautions.

3 Procedures

List of major functions:

Function Module_Subsetimg(**pathname**, **Etg**, **Ntg**, **size_x**, **size_y**)

Function Module_MaskPre(**pathname**, **scpath**, **smpath**)

Function Module_Filter(**pathname**, **tgpathname**, **maskpath**)

Function Module_SpectralIndex(**pathname**, **tgpath**)

Function Module_TimeSeriesStack(**pathname**, **tgpath**, **vegid_x**)

Function Module_Trajectory(**inputdir**, **outputdir**, **yfile**, **lcfile**, **combinelist**, **numparts**, **par**)

Function Module_FittingModels(**imagedir**, **shapedir**, **modeldir**, **yfile**, **combinelist**, **maxsample**, **SVMgsOption**)

Function Module_Classification(**inputdir**, **outputdir**, **modeldir**, **lcfile**, **combinelist**, **numparts**, **algorithm**)

Function Module_CombineMaps(**datadir**, **lcfile**, **combinelist**, **algorithm**)

Function Module_PostProcess(**datadir**, **algorithm**, **MMU**, **neighbor**, **maxiter**)

3.1 Processing subsets

Use function Module_Subsetimg.m

Function Module_Subsetimg(**pathname**, **Etg**, **Ntg**, **size_x**, **size_y**)

Images are required to have the same coordinate system and the subset scene is completely included in all images.

INPUT

pathname is the directory for all images to be subset

Etg and **Ntg** specify the Easting and Northing start coordinates for the subset scene.

size_x and **size_y** specify the size for the subscene

OUTPUT

Subset scenes located in the same folder (**pathname**). They should be moved to another folder before proceeding.

For example, to subset images in "F:\Data\RawImg\" and the start coordinates under UTM coordinate system is 354375 for easting and 4485885 for northing. The size of the subscene is 3100 by 3100. Then type the script as the following:

```
>> Module_Subsetimg('F:\Data\RawImg\', 354375, 4485885, 3100, 3100);
```

An example out is,

Processing 1 of the 2 files...

Opening F:\Data\RawImg\2001179 of 7951cols x 7301rows x 6bands of type int16 image...

Done Reading.

Writing F:\Data\RawImg\2001179s. Processing ...

```
Done Writing.
Processing 2 of the 2 files...
Opening F:\Data\RawImg\2001211 of 7941cols x 7301rows x 6bands of type int16 image...
Done Reading.
Writing F:\Data\RawImg\2001211s. Processing ...
Done Writing.
```

The input files are all files in the directory and the output files add an 's' in the tail. Relocate the output images manually after the subset process. For instance, the output files are moved to 'F:\Data\RefImg\'.

The subsetting process should also be implemented for the cloud mask files.

3.2 Preprocessing Masking Files (Optional)

Use function `Module_MaskPre.m`.

Function `Module_MaskPre(pathname, scpath, smpath)`

This function combines a few mask sources into a single mask file for each subscene. All input images should be of the same size, coordinate systems and geographic extent. These masks are Fmask processed images, study area mask image and a supplemental mask image either manually drawn or automatically generated.

INPUT

pathname is the target output folder, the study area mask is also in this directory and should be named as 'studyarea'. For the study area mask and the supplemental mask, 1 is on and 0 is off and masked.

scpath is the folder that stores the Fmask classified results and files must end with 'fmask'. fmask images will retain the pixel values.

smpath is the folder stores the manual masks and must end with 'sm'. 1 is on and 0 is off and will be masked.

OUTPUT

The combined mask files that end with 'mask' are derived and put into **pathname**.

For instance, the Fmask processed mask images are stored in "F:\Data\CloudFmasks\" folder and all images end with "Fmasks". The study area mask is located in "F:\Data\MaskImg\" folder, which is also the output folder for the combined mask images. The study area mask must be named as "studyarea". The supplemental mask images are stored in "F:\Data\StripeMask\" folder and must end with "sm". This is optional, and when it exists, it should use the same base name as the corresponding Fmask processed the scene. For instance, the Fmask image is "2001179mask", then the supplemental mask image should be named "2001179sm". The study area mask and the supplemental mask should contain only 0 and 1, with 0 to be masked and 1 to be retained.

We use the following script:

```
>> Module_MaskPre('F:\Data\MaskImg\','F:\Data\CloudFmasks\','F:\Data\StripeMask\');
```

```
Opening F:\Data\MaskImg\studyarea of 3100cols x 3100rows x 1bands of type uint8 image...
Done Reading.
```

```

Processing 1 of the 2 files...
Opening F:\Data\CloudFmasks\2001179Fmasks of 3100cols x 3100rows x 1bands of type uint8 image...
Done Reading.
Writing F:\Data\MaskImg\2001179mask. Processing ...
Done Writing.
Processing 2 of the 2 files...
Opening F:\Data\CloudFmasks\2001211Fmasks of 3100cols x 3100rows x 1bands of type uint8 image...
Done Reading.
Opening F:\Data\StripeMask\2001211sm of 3100cols x 3100rows x 1bands of type uint8 image...
Done Reading.
Writing F:\Data\MaskImg\2001211mask. Processing ...
Done Writing.

```

The output combined images are now stored in “F:\Data\MaskImg\” and end with “mask”. There is also an Excel file named ‘stats.xlsx’ generated with format as Time, Water, Shadow, Snow, Cloud and Land in number of pixels:

2001179	57279	298375	237	757900	4264114
2001211	55415	292790	2	587808	4441890

Note: The time listed in stats.xlsx will be used in later processes.

3.3 Filtering and Masking

Use function Module_Filter.m.

Function Module_Filter(**pathname**, **tgpathname**, **maskpath**, **filtering**)

Since the combined masks keep the same value labels as the Fmask processed mask images, thus only values of 5 which indicating land for the mask images retain the pixels of the input images. A 3 by 3 low pass filter is followed by the masking. If the user does not want to apply a mask or filter, leave **maskpath** as “”, or **filtering** as false.

INPUT

pathname is the folder for the surface reflectance images. Files end with ‘s’.

tgpathname is the output folder.

maskpath is the folder where mask images store. Files end with ‘mask’. Leave it as “” if no mask is applied.

filtering: true – a 3 by 3 mean filtering is applied, false – no filtering is done.

OUTPUT

Filtered and masked images are derived from this function and stored in **tgpathname**. The files end with ‘s’.

For example, the surface reflectance images are stored in “F:\Data\RefImg\” and end with ‘s’, while the masked images are stored in “F:\Data\MaskImg\” and end with “mask”. The base name of the two should be identical. For instance, if a scene is named ‘2001179s’ for the surface reflectance image, its mask should be named ‘2001179mask’. We create a folder named “F:\Data\RefImgFil\” to store the post-filtering/masking images.

The script we enter is:


```
>> Module_Filter('F:\Data\RefImg\' , 'F:\Data\RefImgFil\' , 'F:\Data\MaskImg\' , true);
```

```
Opening F:\Data\RefImg\2001179s of 3100cols x 3100rows x 6bands of type int16 image...
Done Reading.
Opening F:\Data\MaskImg\2001179mask of 3100cols x 3100rows x 1bands of type uint8 image...
Done Reading.
Implementing the 3 by 3 low pass filter. Processing...
Writing F:\Data\RefImgFil\2001179s. Processing ...
Done Writing.
Opening F:\Data\RefImg\2001211s of 3100cols x 3100rows x 6bands of type int16 image...
Done Reading.
Opening F:\Data\MaskImg\2001211mask of 3100cols x 3100rows x 1bands of type uint8 image...
Done Reading.
Implementing the 3 by 3 low pass filter. Processing...
Writing F:\Data\RefImgFil\2001211s. Processing ...
Done Writing.
```

The output images should appear in the target folder “F:\Data\RefImgFil\” and end with ‘s’ as the same as the input reflectance images.

3.4 Calculating Spectral Indices

Use function `Module_SpectralIndex.m`.

Function `Module_SpectralIndex(pathname, tgpath)`

Nine spectral indices are automatically calculated from the 6 spectral bands of Landsat TM, which are Band 1, 2, 3, 4, 5 and 7. The spectral indices are NDVI, NDSI, NDWI, NDMI, NBR, EVI, Brightness, Greenness and Wetness (Tasseled Cap).

INPUT

pathname is the folder for the surface reflectance images with 6 spectral bands Band 1, 2, 3, 4, 5, 7 for Landsat TM. Files end with ‘s’.

tgpath is the output folder.

OUTPUT

Output images with additional spectral bands including NDVI, NDSI, NDWI, NDMI, NBR, EVI, Brightness, Greenness and Wetness as spectral indices are generated in **tgpath**. Files end with ‘c’.

For example, the surface reflectance images are stored in “F:\Data\RefImgFil\” and we create a folder named “F:\Data\IdxImg\” to store the output images with spectral indices calculated. The following script will do the work,

```
>> Module_SpectralIndex('F:\Data\RefImgFil\' , 'F:\Data\IdxImg\');
```

```
Processing 1 of the 2 files...
Opening F:\Data\RefImgFil\2001179s of 3100cols x 3100rows x 6bands of type int16 image...
Done Reading.
Writing F:\Data\IdxImg\2001179c. Processing ...
Done Writing.
Processing 2 of the 2 files...
Opening F:\Data\RefImgFil\2001211s of 3100cols x 3100rows x 6bands of type int16 image...
Done Reading.
Writing F:\Data\IdxImg\2001211c. Processing ...
Done Writing.
```

The output files are named with the combination of the base name and 'c'. The result images have 15 bands in total.

3.5 Stacking Time-Series Layers

Use function `Module_TimeSeriesStack.m`.

Function `Module_TimeSeriesStack(pathname, tgpath, vegidx)`

This function can stack time-series layers to form a single time-series image for a specified spectral index.

INPUT

pathname is the folder where there are all images with spectral indices calculated. Files end with 's'
tgpath is the output folder.

vegidx is the index for the bands:

1 - B1 2 - B2 3 - B3 4 - B4 5 - B5 6 - B7 7 - NDVI 8 - NDSI 9 - NDWI 10 - NDMI 11 - NBR
 12 - EVI 13 - Brightness 14 - Greenness 15 - Wetness

OUTPUT

An image to store time-series are generated for the spectral index **vegidx** listed above. The filename consists of index name and '_TS' and is output in **tgpath**.

For instance, images that contain spectral indices are stored in "F:\Data\IdxImg\". The output folder is set to "F:\Data\TimeSeries\". We want to stack NDVI layers and NDVI is number 7 in the list.

Therefore the script line is,

```
>> Module_TimeSeriesStack('F:\Data\IdxImg\','F:\Data\TimeSeries\',7);
```

Processing 1 of the 2 files...

Opening F:\Data\IdxImg\2001179c of 3100cols x 3100rows x 15bands of type int16 image...

Done Reading.

Processing 2 of the 2 files...

Opening F:\Data\IdxImg\2001211c of 3100cols x 3100rows x 15bands of type int16 image...

Done Reading.

Writing F:\Data\TimeSeries\NDVI_TS. Processing ...

Done Writing.

The file "NDVI_TS" is derived as a result. Since there are only 2 images used in the example, the result time-series image has only 2 bands.

In our experiment, we found NDMI, Brightness, NBR, Wetness and NDVI are the best five indices suitable for disturbance detection task between slow-onset and rapid-onset disturbances.

Therefore, we recommend the following commands to derive these results:

```
>> Module_TimeSeriesStack('F:\Data\IdxImg\','F:\Data\TimeSeries\',7);
```

```
>> Module_TimeSeriesStack('F:\Data\IdxImg\','F:\Data\TimeSeries\',10);
```

```
>> Module_TimeSeriesStack('F:\Data\IdxImg\','F:\Data\TimeSeries\',11);
```

```
>> Module_TimeSeriesStack('F:\Data\IdxImg\','F:\Data\TimeSeries\',13);
```

```
>> Module_TimeSeriesStack('F:\Data\IdxImg\','F:\Data\TimeSeries\',15);
```

3.6 Trajectory Extraction

Use Module_Trajectory.m.

Function Module_Trajectory(**inputdir**, **outputdir**, **yfile**, **lcfile**, **combinelist**, **numparts**, **par**)

This function reads time-series images and process into trajectory segments and save into xls Excel file format. In particular, inter-year value selection, noise removal and segmentation are implemented in sequence to yield the trajectory. The function also supports multi-thread execution.

INPUT

inputdir: input folder in which time-series images are named as NDVI_TS, NBR_TS, etc.

outputdir: the folder to store the output trajectories. Files are Excel files with extension ‘.xlsx’.

yfile: the Excel file in which the first column stores the times of acquisition, for instance 2001179, 2002158, 2003207, etc.

lcfile: the forest mask file. 1 is forest, 0 is non-forest. This file must have the same size and coordinate system as the time-series images.

numparts: number of parts in the output. Increase this number to avoid big Excel files and save progressively into the hard drive.

par is the number of workers/threads for parallel CPU processing.

OUTPUT

Trajectory files in Excel format. For each index, the trajectories are stored in two files, a “nodx” and a “nody” file. They together define the breakpoints of the trajectory. Each line indicates a trajectory segments for an unmasked pixel. The “nodx” files and “nody” files are in the format of the following tables.

	A	B	C	D	E		A	B	C	D	E
1	2001	2005	2010	2011		1	4761	5650	3948	3948	
2	2001	2003	2004	2011		2	5007	5037	5743	4045.5	
3	2001	2005	2011			3	4651	4881	3688.5		
4	2001	2002	2004	2011		4	4681	4411	4803	3278.5	
5	2001	2005	2009	2011		5	4965	5092	3169	2999	
6	2001	2005	2009	2011		6	5201	4983	3308	3674	
7	2001	2005	2009	2011		7	5422	5015	4011	4261	
8	2001	2003	2006	2007	2011	8	5141	4836	4994	4559	4418
9	2001	2005	2007	2011		9	4872	4785	4356	4292	
10	2001	2010	2011			10	4835	3725	3725		

For instance, we like to extract trajectories for NDMI, NBR and Wetness, and we have ‘NDMI_TS’, ‘NBR_TS’ and ‘Wetness_TS’ in the folder “F:\Data\TimeSeries\”. The output folder is created as “F:\Data\TrOutput\”. Previously the “stats.xlsx” is automatically generated in the output folder using Module_MaskPre.m. If **3.2 Mask Preprocessing** is not implemented, the file must be manually input with the first column contain time of acquisition in the format of yyyyddd. E.g. 2001179 is year 2001, 179th day. The “stats.xlsx” should have the same number of rows as the bands of the time-series image. We break the output into 4 parts (**numparts**) to save to the hard

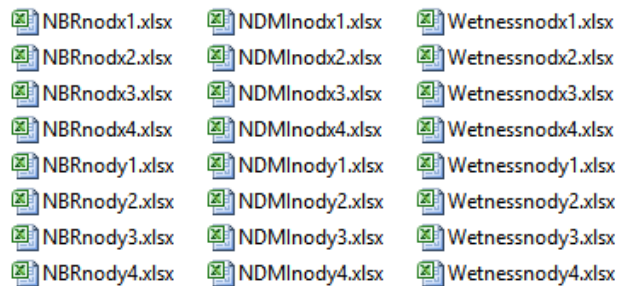
drive from time to time so that if terminated unexpectedly, with minor editing of the script, one can continue the unfinished work without restarting. Normally the trajectory extraction is time consuming.

```
>> combinelist = [10 11 15]; % NDMI NBR Wetness
>> inputdir = 'F:\Data\TimeSeries\';
>> outputdir = 'F:\Data\TrOutput\';
>> yfile = 'F:\Data\stats.xlsx';
>> lcfile = 'F:\Data\forestmask';
>> numparts = 4;
>> cores = 4;
>> Module_Trajectory(inputdir, outputdir, yfile, lcfile, combinelist, numparts, cores);
```

```
Starting matlabpool using the 'local' profile ...
connected to 4 workers.
Processing Trajectory Extraction for NDMI...
Opening F:\Data\forestmask of 400cols x 400rows x 1bands of type uint8 image...
Done Reading.
Opening F:\Data\TimeSeries\NDMI_TS of 400cols x 400rows x 23bands of type int16 image...
Done Reading.
Processing part 1...
writing...
Elapsed time is 47.080310 seconds.
Processing part 2...
writing...
Elapsed time is 80.196413 seconds.
Processing part 3...
writing...
Elapsed time is 97.250794 seconds.
Processing part 4...
writing...
Elapsed time is 100.414248 seconds.
Processing Trajectory Extraction for NBR...
Opening F:\Data\forestmask of 400cols x 400rows x 1bands of type uint8 image...
Done Reading.
Opening F:\Data\TimeSeries\NBR_TS of 400cols x 400rows x 23bands of type int16 image...
Done Reading.
Processing part 1...
writing...
Elapsed time is 57.158349 seconds.
Processing part 2...
writing...
Elapsed time is 82.484238 seconds.
Processing part 3...
writing...
Elapsed time is 97.515980 seconds.
Processing part 4...
writing...
Elapsed time is 103.559883 seconds.
Processing Trajectory Extraction for Wetness...
Opening F:\Data\forestmask of 400cols x 400rows x 1bands of type uint8 image...
Done Reading.
Opening F:\Data\TimeSeries\Wetness_TS of 400cols x 400rows x 23bands of type int16 image...
Done Reading.
Processing part 1...
writing...
Elapsed time is 56.211223 seconds.
Processing part 2...
writing...
Elapsed time is 68.136010 seconds.
Processing part 3...
```

writing...
 Elapsed time is 81.099478 seconds.
 Processing part 4...
 writing...
 Elapsed time is 99.445204 seconds.
 Sending a stop signal to all the workers ... stopped.

As a result, the following files are generated.



“NBRnodx#.xlsx” stores the x-axis value (year) for the breakpoints and “NBRnody#.xlsx” stores the y-axis value (index value). # is the index for parts, and you have **numparts** parts in total for the trajectories of each spectral index.

3.7 Fitting Classification Models (Optional)

Use Module_FittingModels.m

Function Module_FittingModels(**imagedir**, **shapedir**, **modeldir**, **yfile**, **combinelist**, **maxsample**, **SVMgsOption**)

This function extracts values from training samples in shape file (".shp") format and trains SVM model or CART model. Prepare three shape files for persistent forest, slow-onset disturbances and rapid-onset disturbances respectively as training data samples. The samples are in point format. They must be named as 'persistent.shp', 'slowonset.shp' and 'rapidonset.shp'. The shape files should include all files with the same base name, with extension such as ".dbf", ".shx", ".prj", ".sbn" and ".sbx".

For the Support Vector Machine algorithm (SVM), BITE uses libsvm.

The BITE source code package includes libsvm.

Chih-Chung Chang and Chih-Jen Lin, LIBSVM : a library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2:27:1--27:27, 2011.

Software available at <http://www.csie.ntu.edu.tw/~cjlin/libsvm>.

In addition, if the mex files are not compatible with user's current Matlab version, type "make" in the command line to build mex files for the libsvm functions.

INPUT

imagedir is the directory for time-series image.

shapedir is the folder to store 'persistent.shp', 'slowonset.shp' and 'rapidonset.shp' as the training data samples. The shape files are in point format and each point is a sample location. They should have the completely same coordinate system as the images.

modeldir is where the output models are saved.

yfile: the Excel file in which the first column stores the times of acquisition, for instance 2001179, 2002158, 2003207, etc..

maxsample is the largest number of training samples in a single training class. For instance, if the training samples for the three classes have 60, 70 and 65. The **maxsample** should be set to be greater than 70. Conservatively, one can just use the number of training samples here.

combinelist: time-series images of the spectral indices in the list will be processed.

SVMgsOption: if SVMgsOption = [1,2,11,-7,1,5], it means to search C from 2^1 to 2^{13} with step = 2 and to search Gamma from 2^{-7} to 2^5 with step = 1.

OUTPUT

Models and corresponding scaling factors will be saved to **modeldir**. For instance, the files for index NBR are “NBRmaxX.mat”, “NBRminX.mat”, “NBR SVM.mat” and “NBRtree.mat”. The first two files store scaling factors, while the remaining two store the SVM model and CART model respectively.

For example, time-series images are stored in “F:\Data\TimeSeries\”. Training data samples are stored in “F:\Data\ReferencePoints\”. The three files are named as 'persistent.shp', 'slowonset.shp' and 'rapidonset.shp' respectively. “stats.xlsx” is already used in **3.6 Trajectory Extraction** and is also used in this function. We create a folder named “F:\Data\Models\” to store the output models and scaling factors. Since we have 301 training samples in total and the numbers are balanced for the 3 classes, **maxsample** is set to 200. For grid search for the SVM models, we search C from 2^1 to 2^{13} with step = 1 and Gamma from 2^{-7} to 2^5 with step = 1. The following scripts execute the example,

```
>> imagedir = 'F:\Data\TimeSeries\';
>> shapedir = 'F:\Data\ReferencePoints\';
>> modeldir = 'F:\Data\Models\';
>> yfile = 'F:\Data\stats.xlsx';
>> maxsample = 200;
>> combinelist = [10 11 15]; % Still NDMI, NBR and Wetness
>> SVMgsOption = [1,1,13,-7,1,5];
>> Module_FittingModels(imagedir, shapedir, modeldir, yfile, combinelist, maxsample,
SVMgsOption);
```

The output models and scaling factors are now generated in the model folder. The extension for these files is '.mat', which stores matlab variables and struct variables.

 NBRmaxX Microsoft Access Table Shortcut 176 bytes	 NBRminX Microsoft Access Table Shortcut 176 bytes	 NBRsVM Microsoft Access Table Shortcut 4.97 KB
 NBRtree Microsoft Access Table Shortcut 2.93 KB	 NDMImaxX Microsoft Access Table Shortcut 176 bytes	 NDMIminX Microsoft Access Table Shortcut 176 bytes
 NDMISVM Microsoft Access Table Shortcut 4.45 KB	 NDMitree Microsoft Access Table Shortcut 2.76 KB	 WetnessmaxX Microsoft Access Table Shortcut 176 bytes
 WetnessminX Microsoft Access Table Shortcut 176 bytes	 WetnessSVM Microsoft Access Table Shortcut 3.24 KB	 Wetnesstree Microsoft Access Table Shortcut 2.77 KB

3.8 Classifying Images and Integrating Maps

Use Module_Classification.m.

Function Module_Classification(**inputdir**, **outputdir**, **modeldir**, **lcfile**, **combinelist**, **numparts**, **algorithm**)

This function classifies trajectory Excel files and writes 3 disturbance maps for each spectral index using classification models. The models can be generated in **3.7 Fitting Classification Models** or the user can use default models provided with the BITE code package, which were proven to achieve an overall accuracy around 95% (After integrating the Brightness, NDMI, NBR, Wetness and NDVI maps) for disturbance type detections in Grand County, Colorado. The test results are presented the table below the paragraph. However, it must be noted that the generalization of the default models has not been experimented to other areas yet. Use with cautions.

Spectral Indices	CART 5-CV	CART Test	SVM 5-CV	SVM Test
NDVI*	95.3%	88.2%	94.0%	90.9%
NDSI	89.7%	73.9%	98.0%	73.8%
NDWI	94.0%	85.4%	94.0%	90.4%
NDMI*	95.7%	87.5%	95.0%	92.4%
NBR*	95.7%	88.5%	95.3%	92.2%
EVI	90.7%	70.4%	84.1%	78.7%
Brightness*	95.0%	88.9%	93.4%	92.4%
Greenness	93.0%	83.6%	86.0%	84.2%
Wetness*	96.7%	89.4%	96.3%	91.5%

* Indices were selected for integration.

'CV' represents the cross-validation test on the training dataset.

'Test' represents the evaluation on the test dataset.

INPUT

inputdir: folder for trajectory Excel files.

outputdir: folder to store the output disturbance maps.

modeldir: folder for model files. For NDMI to do classification using SVM, one must have "NDMIminX.mat", "NDMImaxX.mat" and "NDMISVM.mat". To use CART, only "NDMitree.mat" is required.

lcfile: the complete filename for the forest mask.

combinelist: the array to store the list of spectral indices to process in sequence.

numparts: the same number as being used in Module_Disturb.m.

algorithm: 1 – CART, 2 – SVM.

OUTPUT

The output include a type map, a slow-onset disturbance map and a rapid-onset disturbance map generated in **outputdir**. For the type map, pixel values are defined as the following: 0 - unclassified, 1 – persistent, 2 - slow-onset, 3 - rapid-onset.

For instance, to produce disturbance maps for NDMI, NBR and Wetness using SVM classification algorithm, the following procedure is recommended. The trajectory Excel files are in the folder “F:\Data\TrOutput\”. The output folder is created as “F:\Data\DistMap\”. Matlab model files are stored in “F:\Data\Models\”. And the SVM model files are “NDMIminX.mat”, “NDMImaxX.mat” and “NDMISVM.mat”. The forest mask file is “F:\Data\forestmask”. We use the following scripts,

```
>> inputdir = 'F:\Data\TrOutput\';
>> modeldir = 'F:\Data\Models\';
>> outputdir = 'F:\Data\DistMap\';
>> lcfile = 'F:\Data\forestmask';
>> combinelist = [10 11 15]; % NDMI, NBR and Wetness
>> numparts = 4;
>> algorithm = 2; % 1 - CART 2 - SVM
>> Module_Classification(inputdir, outputdir, modeldir, lcfile, combinelist, numparts, algorithm);
```

Opening F:\Data\forestmask of 400cols x 400rows x 1bands of type uint8 image...

Done Reading.

Accuracy = 0% (0/33884) (classification)

Accuracy = 0% (0/33885) (classification)

Accuracy = 0% (0/33884) (classification)

Accuracy = 0% (0/33885) (classification)

Elapsed time is 13.328390 seconds.

Writing F:\Data\DistMap\SVMNDMItypemap. Processing ...

Done Writing.

Writing F:\Data\DistMap\SVMNDMIslowmap. Processing ...

Done Writing.

Writing F:\Data\DistMap\SVMNDMIslowmap. Processing ...

Done Writing.

Accuracy = 0% (0/33884) (classification)

Accuracy = 0% (0/33885) (classification)

Accuracy = 0% (0/33884) (classification)

Accuracy = 0% (0/33885) (classification)

Elapsed time is 13.286565 seconds.

Writing F:\Data\DistMap\SVMNBRtypemap. Processing ...

Done Writing.

Writing F:\Data\DistMap\SVMNBRslowmap. Processing ...

Done Writing.

Writing F:\Data\DistMap\SVMNBRslowmap. Processing ...

Done Writing.

Accuracy = 0% (0/33884) (classification)

Accuracy = 0% (0/33885) (classification)

Accuracy = 0% (0/33884) (classification)

Accuracy = 0% (0/33885) (classification)

Elapsed time is 13.062449 seconds.


```

Writing F:\Data\DistMap\SVMWetnesstypemap. Processing ...
Done Writing.
Writing F:\Data\DistMap\SVMWetnessslowmap. Processing ...
Done Writing.
Writing F:\Data\DistMap\SVMWetnessrapidmap. Processing ...
Done Writing.

```

Disregard the accuracy information shown in the results because no real validation is done here. Three preliminary disturbance maps are now created for each index. For instance, NDMI has “SVMNDMItypemap” for disturbance types, “SVMNDMIslowmap” for onset years of slow-onset disturbances and “SVMNDMIrapidmap” for onset years of rapid-onset disturbances. Particularly for the type map, 0 is unclassified (masked), 1 is persistent forest, 2 is slow-onset disturbance and 3 is rapid-onset disturbance.

3.9 Integrating Maps

Use Module_CombineMaps.m.

Function Module_CombineMaps(**datadir**, **lcfile**, **combinelist**, **algorithm**)

Since a number of disturbance maps can be generated, each using a different spectral indices. This function integrates these maps into one single disturbance map, potentially enhancing the overall accuracy of the disturbance map.

INPUT

datadir: The folder that disturbance maps are stored.

lcfile: the complete filename for the forest mask.

combinelist: the array list of spectral indices to integrate.

algorithm: 1 – CART, 2 – SVM.

OUTPUT

The output disturbance map is named either “SVMdistmap” or “CARTdistmap” based on whether algorithm is used. The file is stored in datadir. For the pixel values, 0 is masked, 1 is persistent forest, while other values are the onset years of disturbances. For rapid-onset disturbances, the values are onset year + 1000.

For example, the disturbance maps for NBR, NDMI and Wetness using SVM are calculated and stored in “F:\Data\DistMap\”. We would like to integrate the three disturbance maps. The following scripts are recommended.

```
>> combinelist = [10 11 15];
```

```
>> Module_CombineMaps('F:\Data\DistMap\', 'F:\Data\forestmask', combinelist, 2);
```

```

Opening F:\Data\forestmask of 400cols x 400rows x 1bands of type uint8...
Done Reading.
Opening F:\Data\DistMap\SVMNDMItypemap of 400cols x 400rows x 1bands of type uint8...
Done Reading.

```

```

Opening F:\Data\DistMap\SVMNDMIslowmap of 400cols x 400rows x 1bands of type int16...
Done Reading.
Opening F:\Data\DistMap\SVMNDMIsrapidmap of 400cols x 400rows x 1bands of type int16...
Done Reading.
Opening F:\Data\DistMap\SVMNBRtypemap of 400cols x 400rows x 1bands of type uint8...
Done Reading.
Opening F:\Data\DistMap\SVMNBRslowmap of 400cols x 400rows x 1bands of type int16...
Done Reading.
Opening F:\Data\DistMap\SVMNBRrapidmap of 400cols x 400rows x 1bands of type int16...
Done Reading.
Opening F:\Data\DistMap\SVMWetnesstypemap of 400cols x 400rows x 1bands of type uint8...
Done Reading.
Opening F:\Data\DistMap\SVMWetnessslowmap of 400cols x 400rows x 1bands of type int16...
Done Reading.
Opening F:\Data\DistMap\SVMWetnessrapidmap of 400cols x 400rows x 1bands of type int16...
Done Reading.
Writing F:\Data\DistMap\SVMdismap. Processing ...
Done Writing.

```

The result is SVMdismap in "F:\Data\DistMap\".

3.10 Post-Classification Processing

Use Module_PostProcess.m.

Function Module_PostProcess(**datadir**, **algorithm**, **MMU**, **neighbor**, **maxiter**)

In the post-classification processing, firstly a minimum mapping unit (**MMU**) filter is applied to remove connected pixel groups with less than a certain area, which is then followed by refilling using a majority filter to fill the gaps proportionally to surrounding pixels. The final products contain one complete map with all information stored in the same coding approach as the input integrated map, and three separate maps: a type map, a slow-onset disturbance map and a rapid-onset disturbance map.

INPUT

datadir: The folder that disturbance maps are stored.

algorithm: 1 – CART, 2 – SVM.

MMU: The size of the MMU in number of pixels. FIA standards specify one acre (4047 m²) as the MMU for forestland. Therefore, 5 Landsat pixels (4500 m²) should be enough.

neighbor: Specify whether use 4-connected neighbors (Castle) or 8-connected neighbors (Queen). Only 4 or 8 is allowed.

maxiter: The maximum number of passes allowed for the majority filter. We set to 500 normally.

OUTPUT

Output files are all in **datadir** :

One final stand-alone disturbance map. File ends with "dismappost". For the pixel values, 0 is masked, 1 is persistent forest, while other values are the onset years of disturbances. For rapid-onset disturbances, the values are onset year + 1000.

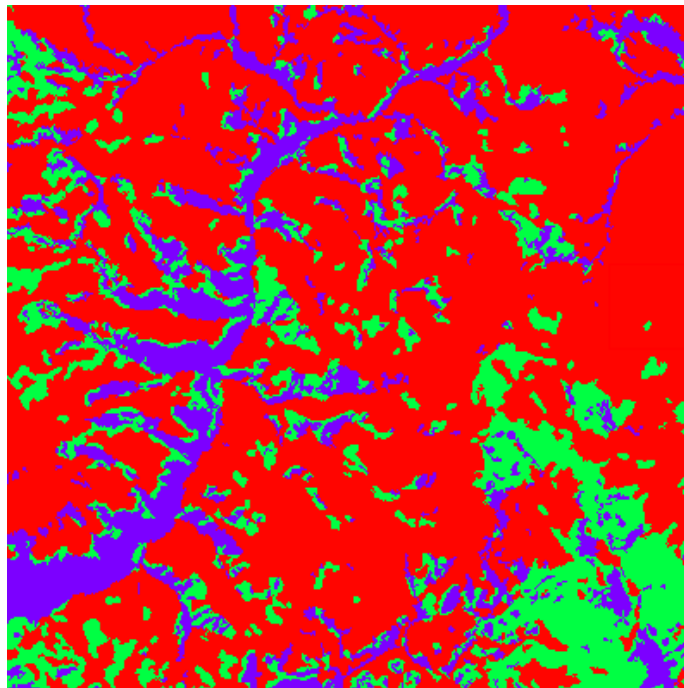
Three separate disturbance maps. "finaltypemap" is the type map, in which 0 - unclassified, 1 – persistent, 2 - slow-onset, 3 - rapid-onset. "finalslowmap" is the map with the onset years for slow-onset disturbances. "finalrapidmap" is the map with the onset years for rapid-onset disturbances.

For instance, we have the integrated map using SVM in “F:\Data\DistMap\”. We would like to carry out the MMU filtering with the size of MMU of 6 with 8-connected neighbors. The following script is recommended,

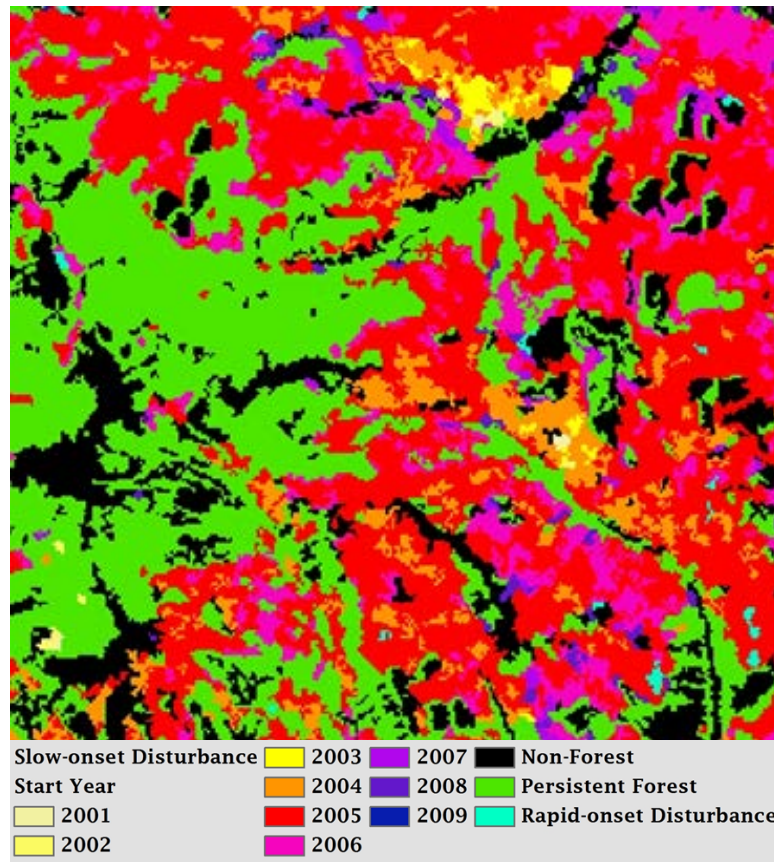
```
>> Module_PostProcess('F:\Data\DistMap\',2, 6, 8, 500);
```

```
Opening F:\Data\DistMap\SVMdistmap of 400cols x 400rows x 1bands of type int16...  
Done Reading.  
Writing F:\Data\DistMap\SVMdistmappost. Processing ...  
Done Writing.  
Writing F:\Data\DistMap\SVMfinaltypemap. Processing ...  
Done Writing.  
Writing F:\Data\DistMap\SVMfinalslowmap. Processing ...  
Done Writing.  
Writing F:\Data\DistMap\SVMfinalrapidmap. Processing ...  
Done Writing.
```

The result type map can be shown in any image processing software. A type map as the result is presented below, in which Blue is non-forest, Green is persistent forest and Red is slow-onset disturbances:



This is the final step of the procedure and the disturbance maps are generated. Onset years of either slow-onset or rapid-onset disturbances are useful in making maps of showing the dispersal pattern of a natural insect or disease outbreak of the trees (slow-onset disturbances) or tracking location and time of the clearcut or wildfires (rapid-onset disturbances). With the help of these disturbance maps generated using BITE, users are capable of doing research on the forest change patterns and exploring the human factors behind the changes for better policy making and management.



4 Acknowledgement

We thank Chih-Chung Chang and Chih-Jen Lin, the authors of LIBSVM, and Damien Garcia, the author of label.m for sharing their codes for public access. We also thank gnovice from <http://stackoverflow.com/questions/2652630/how-to-get-all-files-under-a-specific-directory-in-matlab> to share getAllFiles.m function. We are grateful to the U.S. Geological Survey, Climate and Land Use Mission Area Land Remote Sensing Program provided funding to support the development of this algorithm and relevant research (grant number G12AC20085). Any use of trade, firm, or product names is for descriptive purposes only and does not imply endorsement by the U.S. Government.